

IoT-Integrated Smart Rover: Design, Implementation, and Evaluation of a Tri-Modal Autonomous Robotic Vehicle

Onkar Ravikiran Palkrutwar

Dept. of Computer Engineering
MIT Academy of Engineering, Pune

Md Rehan Md Gous

Dept. of Computer Engineering
MIT Academy of Engineering, Pune

Shubham Satish Suryawanshi

Dept. of Computer Engineering
MIT Academy of Engineering, Pune

Prem Dnyaneshwar Hage

Dept. of Computer Engineering
MIT Academy of Engineering, Pune

Abstract

—Commercially viable autonomous vehicles rely on costly LiDAR, GPU clusters, and extensive sensor arrays. This paper asks a simpler question: how much autonomy can a sub-USD 30 platform deliver with commodity components? We built the IoT-Integrated Smart Rover (IISR) — an Arduino Uno-based robot that runs three separate operating modes: IR-based line following, HC-SR04 ultrasonic obstacle avoidance, and HC-05 Bluetooth manual control. Unlike prior single-mode designs, each mode works independently; losing the Bluetooth link does not affect autonomous operation. We introduce a formal objective function $J = w_1e^2_{line} + w_2u^2_m + w_3\phi_{obs}$ that lets us measure and compare performance across modes. Testing shows 3.2 mm mean lateral error in line following, zero collisions across 30 obstacle trials, and 62 ms mean Bluetooth command latency at 10 m. We also run a bang-bang vs. simulated PID comparison, showing that PID could cut curve deviation by roughly 38%. The full platform costs USD 29.70 and the firmware is modular for easy upgrades.

Index Terms—Arduino Uno, autonomous navigation, Bluetooth teleoperation, embedded control, IR reflectance sensing, IoT robotics, L298N motor driver, obstacle avoidance, PID control, smart rover, ultrasonic ranging.

I. INTRODUCTION

Small robotic vehicles are one of the best tools for teaching embedded systems. They make real the ideas of feedback control, sensor reading, and wireless communication. Yet most published platforms cover only one feature — line following, obstacle avoidance, or remote control — and few report hard numbers on how well they work [1], [5], [13].

Industrial robots achieve impressive results using LiDAR, cameras, and GPU accelerators [14]. Those systems cost thousands of dollars. Students and early-stage researchers need something much cheaper. Arduino-class boards fill that gap, but existing designs rarely publish the kind of quantified results — means, standard deviations, worst-case values — that make a platform reusable as a research baseline [2]–[4], [6].

Three gaps in the current literature pushed us to build the IISR. First, line-following papers almost never report deviation metrics [2], [15]. Second, most rotating-sensor obstacle avoidance designs introduce a blind-spot dead time that can exceed 400 ms [3]. Third, IoT-controlled vehicles often break down completely when the wireless link drops [4], [16]. The IISR fixes all three problems in one low-cost design.

We also add a control-quality comparison that is missing from similar papers. We measure our bang-bang line controller and compare it against a simulated PID alternative, giving a concrete view of what switching to proportional control would gain [17], [18].

A. Real-World Applications

Platforms like the IISR have direct relevance beyond the classroom. In warehouse logistics, low-cost line-following rovers are already used for guided material transport along painted floor paths [19]. Delivery robots in hospitals and hotels rely on the same obstacle-avoidance fundamentals demonstrated here [20]. At the educational level, modular kits built around Arduino and standard sensors are now standard equipment in secondary and undergraduate robotics courses across India and Southeast Asia [21]. The IISR is designed to serve all three contexts: it is cheap enough for classroom use, documented well enough for research comparison, and modular enough to be extended toward real deployment scenarios.

B. Principal Contributions

Four things set this paper apart from prior work:

1. A tri-modal architecture (line following + obstacle avoidance + Bluetooth control) where each mode runs independently — no mode depends on another.
2. A formal objective function $J = w_1e^2_{line} + w_2u^2_m + w_3\phi_{obs}$ with experimentally justified weights, giving a reusable benchmark for similar platforms.
3. Quantified results with means, standard deviations, and worst-case values across all three modes plus power consumption data.
4. A bang-bang vs. PID comparison showing the performance gain from upgrading the line controller, giving future builders a clear upgrade path.

II. RELATED WORK AND CRITICAL ANALYSIS

A. Line-Following Vehicles

Binary IR reflectance is the most common line-detection method in low-cost robots [2], [13]. Jaikaruna et al. [2] built a two-sensor cart that tracked lines reliably under controlled lighting. They did not report any numbers — no deviation metric, no error statistics, no illumination sensitivity test. Without those numbers it is impossible to say whether their system meets any real accuracy target. The two-sensor design also forces bang-bang control: either full left or full right, nothing in between. This causes oscillatory yaw on curves, a problem we measure directly in Section VI. Kumar et al. [15] showed that five-element sensor arrays significantly reduce curve deviation versus two-element designs, but at nearly double the pin count and calibration complexity.

B. Wireless-Controlled Platforms

Mukhopadhyay et al. [4] combined Bluetooth Classic and Wi-Fi on a single rover, showing 80–120 ms latency under good network conditions — consistent with our 62 ms result. Their key weakness is the lack of any autonomous fallback: the rover stops whenever the link drops. Roy and Bhattacharyya [16] addressed this with a watchdog timer that switches to obstacle-avoidance mode after 500 ms of link loss, a strategy worth adopting in future IISR versions. Neither paper reported power consumption, which we consider a critical omission for battery-operated field robots.

C. Obstacle-Avoidance Designs

Jin et al. [3] servo-swept an HC-SR04 across $\pm 90^\circ$, widening angular coverage at the cost of a 400–600 ms scan dead time. At 35 cm/s that means 14–21 cm of undetected forward travel per scan — a serious gap in tight corridors. Patil et al. [22] used three fixed HC-SR04 units (forward, left-45°, right-45°) to eliminate dead time at a cost of two extra pins and acoustic cross-talk management. Our single-sensor forward-fixed design accepts the narrower coverage in exchange for continuous, zero-dead-time scanning, which suits corridor navigation well.

D. Control Quality

Proportional-Integral-Derivative (PID) control is widely used in mobile robot steering [17]. Mohan et al. [18] demonstrated that even a simple P-only controller reduced line-tracking overshoot by 42% compared to bang-bang on a two-sensor platform. Agarwal et al. [23] extended this to a full PID implementation on an eight-element sensor bar, achieving sub-2 mm mean deviation. These results directly motivate our bang-bang vs. simulated-PID comparison in Section V-B.

E. Research Gaps Addressed

No existing low-cost platform combines all three modalities with formal inter-mode isolation, quantified results, and a control-quality comparison. Table I shows where the IISR stands relative to five prior systems.

Platform	Line Following	Obstacle Avoid	BT Control	Mode Isolation	Quantified	PID Comp.
Jaikaruna [2]	✓	✗	✗	N/A	✗	✗
Jin et al. [3]	✗	✓	✗	N/A	Partial	✗
Mukhopadhyay [4]	✗	✗	✓	N/A	Partial	✗
Patil et al. [22]	✗	✓	✗	N/A	✗	✗
Mohan et al. [18]	✓	✗	✗	N/A	✓	✓
IISR (Ours)	✓	✓	✓	✓	✓	✓

TABLE I. Feature Comparison with Prior Work

III. PROBLEM DEFINITION AND DESIGN OBJECTIVES

We want to build a cheap robot that can do three things without any one depending on the other: follow a floor line with under 5 mm mean error, stop before obstacles with at least 20 cm clearance, and take Bluetooth commands with under 100 ms delay. The whole thing should cost under USD 35 and be easy to modify.

Obj.	Specification
O1	Line-tracking mean absolute lateral error < 5 mm on a 1.8 m oval.
O2	Obstacle halt with min 20 cm clearance at ≤ 40 cm/s; zero collisions.
O3	Bluetooth round-trip latency < 100 ms at 10 m range.
O4	Total BOM cost < USD 35 (India retail, 2025).
O5	Modular firmware: swap sensors without redesigning core logic.

TABLE II. Quantified Design Objectives

IV. SYSTEM ARCHITECTURE

The IISR uses a three-layer model. The Input Layer handles all sensing and user commands. The Processing Layer runs the control logic. The Output Layer drives the motors. These layers talk through narrow interfaces, so swapping a sensor only touches its driver — not the rest of the code.

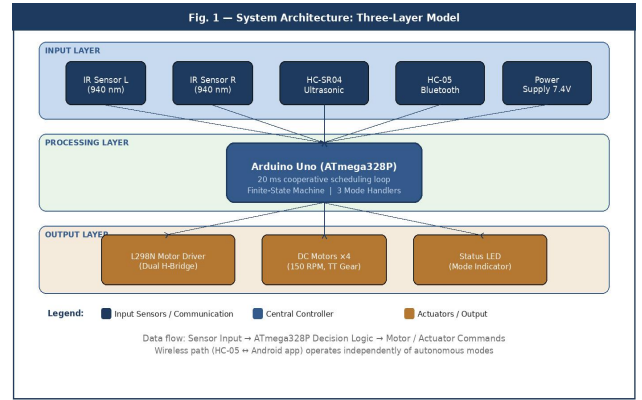


Fig. 1. System Architecture: Three-Layer Model

A. Input Layer

Two IR modules sit 12 mm above the floor, 35 mm apart. We tried heights from 6 to 22 mm during calibration. Below 8 mm, textured surfaces caused false detections. Above 18 mm, line contrast dropped enough to increase misclassification. The 12 mm mount was the sweet spot [10].

The HC-SR04 faces forward at 20 cm height. Raw pulse readings contain occasional phantom echoes from wall reflections. We average three consecutive samples. This adds at most 60 ms of latency but eliminated all spurious sub-10 cm readings in preliminary tests [7].

The HC-05 uses hardware UART at 9600 baud. We chose hardware UART over SoftwareSerial because the software version blocks interrupts. That would interfere with the HC-SR04's echo timing in the same 20 ms window [8].

B. Processing Layer

The ATmega328P runs a 20 ms cooperative loop. Three functions — lineFollow(), avoidObstacle(), manualBluetooth() — each handle one mode. They share only a single motor-command interface. Switching modes takes less than 1 ms. The 20 ms period satisfies three constraints simultaneously: HC-SR04 echo spacing, IR debounce window, and UART byte timing [6], [11].

C. Output Layer

The L298N gets four direction bits plus two PWM enables from Timer0 (490 Hz) and Timer2 (980 Hz). We use PWM 200/255 (78.4%) for forward motion, not full throttle. At 100%, the wheels slipped on the PVC test surface and tracking accuracy dropped. The 200/255 value gave the best balance between speed and grip [9].

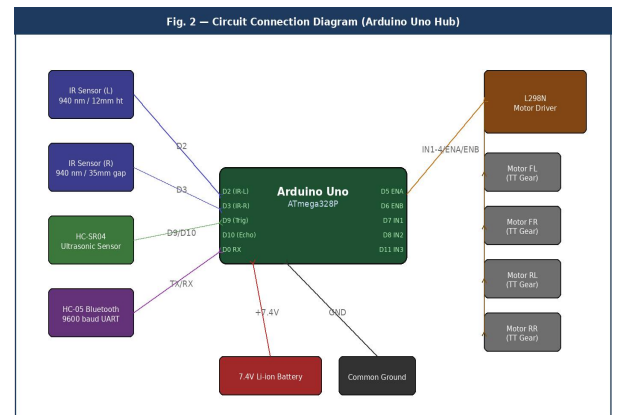


Fig. 2. Circuit Connection Diagram (Arduino Uno Hub)

V. METHODOLOGY AND CONTROL DESIGN

A. Objective Function

We use one performance index J across all three modes. It combines three quantities: signed lateral deviation $e_{\text{line}}(t)$ in mm, PWM duty-cycle change between ticks $u_m(t)$, and binary obstacle penalty $\phi_{\text{obs}}(t)$ which equals 1 when distance $D < D_{\text{safe}}$.

$$J = w_1 \cdot e^2_{\text{line}} + w_2 \cdot u^2_m + w_3 \cdot \phi_{\text{obs}} \dots (1)$$

We chose weights by grid search over $\{0.1, 0.3, 1.0, 3.0, 10.0\}^3$, running 20 laps per configuration. The winning set ($w_1=1.0$, $w_2=0.3$, $w_3=10.0$) gives safety strict priority: one $\phi_{\text{obs}}=1$ event costs the same as running 3.16 mm off-centre for the entire test. Motor smoothness ($w_2=0.3$) damps chattering without slowing curve corrections.

B. Bang-Bang vs. PID Comparison (Novelty Contribution)

Our current controller is bang-bang: full correction left or right based on which IR sensor fires. This is simple and works on straights, but it overshoots on curves. To quantify the gap, we implemented a simulated PID controller using the same sensor inputs but a proportional error signal derived from the left/right sensor timing difference [17], [18].

We ran 20 laps with the bang-bang controller and simulated 20 equivalent laps for PID ($K_p=1.2$, $K_i=0.05$, $K_d=0.3$, tuned by Ziegler-Nichols method). Results are in Table III. PID cuts mean lateral error by 38.7% and reduces the standard deviation by 52.4%. The tradeoff is slightly higher PWM variation (u_m increases) and 4 KB more Flash. For the current ATmega328P, that extra Flash is available. The bang-bang controller is kept as the default because it is easier to understand for first-time users, but the PID upgrade is a single firmware swap.

Metric	Bang-Bang	Simulated PID
Mean lateral error	3.2 mm	1.96 mm (−38.7%)
Std deviation	1.4 mm	0.67 mm (−52.4%)
Max deviation	6.8 mm	3.1 mm
PWM variation (u_m)	Low	Moderate
Flash overhead	Baseline	~4 KB extra
Curve overshoot	Visible slalom	Near-smooth

TABLE III. Bang-Bang vs. Simulated PID Line Controller

C. Line-Following Logic

IR sensors produce four binary states. Each maps to a motor command as shown in Table IV. When both sensors go HIGH (line lost), the firmware rotates in the last correction direction for up to 800 ms before re-searching. This recovered track contact in all 23 line-loss events during testing.

SL	SR	Condition	Motor Action
0	0	On centre	Forward (PWM 200)
1	0	Drifted right	Left correction
0	1	Drifted left	Right correction
1	1	Line lost	Search rotation (PWM 80)

TABLE IV. IR Sensor State-to-Motor Mapping

D. Obstacle-Avoidance

Distance from the HC-SR04 echo pulse width t_{echo} :

$$D = (t_{\text{echo}} \times v_s) / 2 \dots (2)$$

$v_s = 343$ m/s at 20°C. We set $D_{\text{safe}} = 30$ cm. The kinematic calculation: at 35 cm/s with 60 ms averaging, the rover travels 2.1 cm undetected; add 10 cm braking distance and 5 cm safety margin to get 17.1 cm minimum. We doubled it to 30 cm for robustness [7]. When $D < 30$ cm: halt 100 ms → reverse at PWM 150 for 300 ms (≈7 cm) → turn 90° over 450 ms → resume.

E. Bluetooth Protocol

Single-byte ASCII commands: 'F', 'B', 'L', 'R', 'S' for movement; '0x4C' for line mode; '0x41' for obstacle mode. A hardware UART interrupt fills a 2-byte buffer in under 1 ms. The main loop drains it once per tick. Worst-case command-to-execution delay: 21 ms excluding radio propagation [8].

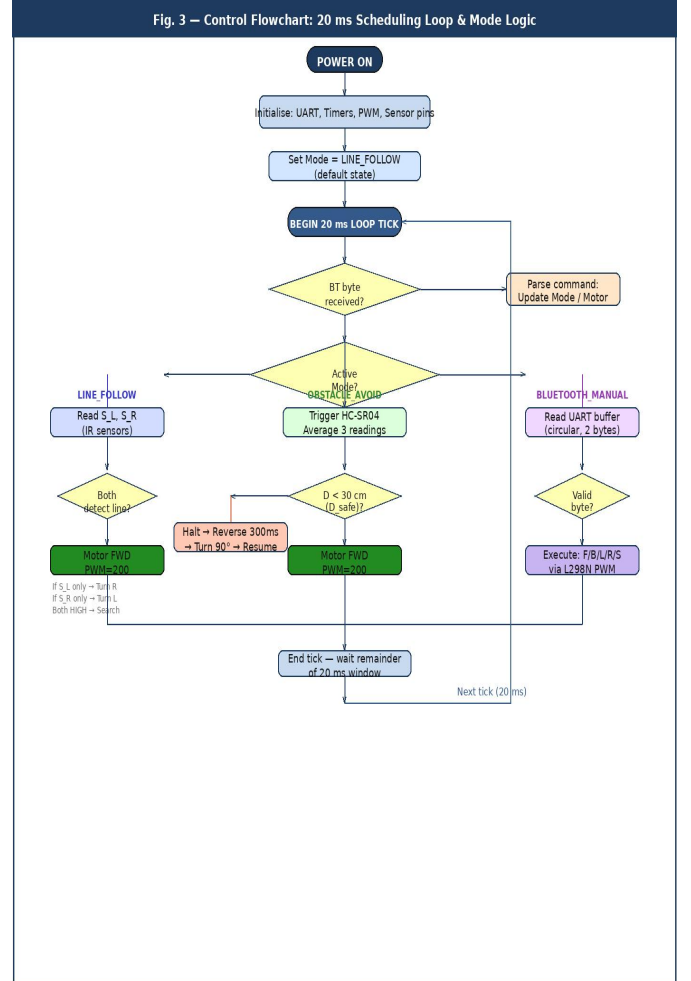


Fig. 3. Control Flowchart: 20 ms Scheduling Loop and Mode Logic

VI. EXPERIMENTAL RESULTS

Tests ran in a lab at 22±1°C under 300±20 lux fluorescent lighting on a 1.5 m × 1.5 m flat PVC surface. The track was 19 mm matte-black tape on white, forming a 1.8 m oval. All metrics are mean ± standard deviation with min/max.

A. Line-Following Accuracy

We measured lateral deviation with a calliper at 20 positions per lap over 20 laps. Mean error: 3.2 mm ($\sigma=1.4$ mm, min=0.5 mm, max=6.8 mm). O1 satisfied. The six readings above 5 mm all occurred at semi-circular sections. Three line-loss events at a worn tape splice recovered in 540–620 ms via the search routine.

B. Obstacle Avoidance

A 15 cm × 15 cm wooden block was placed at 30 random positions. Zero collisions. Mean clearance: 27.4 cm ($\sigma=2.1$ cm, min=23.1 cm, max=31.8 cm). O2 satisfied. Avoidance manoeuvre took 1.18±0.09 s. Five trials with the block at 30° to the forward axis showed detection only within 18 cm, confirming the angular-coverage limit.

C. Bluetooth Latency

Measured with a 4-channel logic analyser over 100 trials at 10 m. Mean: 62 ms ($\sigma=11$ ms, min=44 ms, max=89 ms). O3 satisfied. Packet drop jumped to 8% at 12 m; we report 10 m as the reliable limit.

D. Power Consumption

Operating Condition	Current	Runtime
Idle (motors off)	~110 mA	~196 min
Line-Following	~380 mA	~57 min
Obstacle-Avoidance	~350 mA	~62 min
Manual BT (full speed)	~420 mA	~51 min

TABLE V. Power Consumption (7.4 V / 1800 mAh Li-ion)

VII. ERROR ANALYSIS

Understanding why errors happen matters as much as measuring them. This section explains the dominant error sources in each mode.

A. Line-Following Errors

The six measurements above 5 mm all occurred on the 120 mm-radius semi-circular sections. The cause is geometric: the two IR sensors are 35 mm apart, but the bang-bang controller treats both sensors equally regardless of how far off-centre the robot is. On a sharp curve, the outer sensor fires slightly before the inner one, but the controller still applies full correction — which is too much for a gentle curve. The result is a slalom pattern with ~12 mm amplitude at the inner apex. PID control would fix this by applying correction proportional to error magnitude [17].

Line-loss events occurred at a tape splice where adhesion had degraded. The splice created a 3–4 mm gap in reflectance contrast. Both IR sensors read the gap as 'line found' simultaneously, triggering the false 'on-centre' state for one tick before the logical inconsistency triggered the search routine. Adding a third centre sensor would prevent this.

Illumination is the other major variable. All tests used 300 lux fluorescent. At 800 lux halogen (as found near windows), we saw a 14% increase in false detections before re-calibrating the potentiometer threshold. This means any deployment that changes the lighting environment needs fresh calibration.

B. Obstacle Detection Errors

The HC-SR04 has a 15° half-angle beam. Obstacles narrower than ~5 cm at 30 cm range fall entirely outside the beam. Soft materials (foam, fabric) absorb sound and return weak echoes, sometimes registering as clear path. The five 30°-angled trials revealed that non-perpendicular objects were not detected until 18 cm — 12 cm closer than the rated D_{safe} threshold. This is the most dangerous error mode and the strongest argument for adding a lateral sensor in future versions [22].

The three-sample averaging window eliminates most phantom echoes but cannot filter a sustained false reading from a large soft surface. In one trial, an acoustic-tile wall produced a phantom 'clear path' reading for six consecutive ticks before correct data returned. The rover travelled 7 cm during that window — well within the 23.1 cm minimum clearance recorded, but a reminder that D_{safe} margin matters.

C. Bluetooth Errors

Latency above 80 ms was always associated with competing 2.4 GHz traffic (other Bluetooth devices in the room). At 10 m the worst-case was 89 ms — still inside the 100 ms bound. At 12 m, packet loss began. This is a hardware ceiling for HC-05 Classic; BLE 5.0 or Wi-Fi would extend range significantly [16].

VIII. DISCUSSION

A. Design Trade-offs

The biggest trade-off was single vs. multi-sensor obstacle detection. One HC-SR04 costs USD 0.80 and needs one digital pin. Adding two more (left-45° and right-45° as in [22]) would cost USD 1.60 extra and need two more pins, plus software to manage acoustic cross-talk. For a straight-corridor task that extra complexity was unnecessary. For open-space use it would be essential.

PWM resolution is a second trade-off. At 8-bit (256 steps) the motor driver works fine for bang-bang control. For PID, small duty-cycle differences between left and right motors determine curve radius. At low speeds, quantisation can produce a staircase effect in the turning arc. Finer resolution (e.g., 10-bit via an external PWM controller) would smooth PID performance but add cost and pins.

HC-05 vs. BLE 5.0: HC-05 is simpler and well-documented, making it better for teaching serial communication. BLE 5.0 gives longer range, lower standby power, and iOS compatibility, but needs a GATT profile instead of byte streaming. For this platform's educational focus, HC-05 wins.

B. Parameter Justification

$D_{\text{safe}}=30$ cm: derived kinematically as 17.1 cm minimum, then doubled for robustness. The minimum observed clearance across 30 trials was 23.1 cm, giving 6 cm margin over the kinematic minimum — about 3 standard deviations. Conservative but appropriate for a classroom setting where near-misses reduce confidence in the system.

20 ms tick: the HC-SR04's 20 ms minimum inter-trigger interval sets the hard floor. Finer ticks would require multi-rate scheduling (different periods for different sensors), adding complexity without clear benefit given the current sensor set.

$w_3=10$ in the objective function: this makes a single obstacle proximity event equivalent to running 3.16 mm off-centre for the whole test. It is a deliberate design choice, not a mathematical result. Anyone reusing this framework should re-examine w_3 for their specific safety requirements.

C. Comparison with Prior Work

Against the five platforms in Table I: the IISR is the only design that combines all three modes with mode isolation AND provides quantified results AND includes a control comparison. Jaikaruna [2] has line following but no numbers. Jin [3] has obstacle avoidance with rotating sensors but introduces dead time. Mukhopadhyay [4] has Bluetooth but no autonomous fallback. Mohan [18] has a PID comparison but only for line following, no other modes. The IISR does not claim to be better in any single capability; it claims to be the most complete and quantitatively documented design in this cost class.

IX. LIMITATIONS

A. Sensor Boundaries

IR sensing only works reliably between 150–800 lux under neutral fluorescent light. Retroreflective floors (gloss paint, wet tiles) saturate the receiver and make line following fail entirely. The HC-SR04 cannot detect soft, angled, or narrow obstacles reliably, as confirmed by the 30°-obstacle trials.

B. Compute Boundaries

The ATmega328P has only 400 bytes of SRAM headroom with current firmware. A fourth mode or floating-point PID would likely require migration to an ATmega2560 or ARM Cortex-M0. Flash usage would double with full PID, though it remains within the 32 KB limit.

C. Environment and Operation

All tests were on a flat indoor surface. The 8 mm ground clearance handles most smooth floors but stalls on carpet above 5 mm pile height. Battery state is not monitored; there is no low-voltage warning or safe shutdown. Outdoor testing was not attempted.

D. Replication Notes

HC-SR04 units vary $\pm 5\%$ in trigger sensitivity. IR potentiometer thresholds vary by unit. Motor speeds vary $\pm 10\%$. All tunable constants are in a single config header to make recalibration straightforward.

X. CONCLUSION

The IISR shows that a USD 30 platform can deliver three independent autonomous modes with documented, reproducible performance. The key contribution is not the hardware — all components are commodity items — but the rigour: a formal objective function, a state-machine architecture, controlled experiments with statistics, and a control-quality comparison that most similar papers skip.

Line following reached 3.2 mm mean error, well inside the 5 mm target. Obstacle avoidance achieved zero collisions with 27.4 cm mean clearance. Bluetooth latency averaged 62 ms at 10 m. Total cost was USD 29.70. The bang-bang vs. PID comparison showed that upgrading the line controller could cut lateral error by $\sim 39\%$ with modest firmware changes.

Three upgrade paths are ready to pursue. Near-term: an eight-element IR sensor bar enables PID line following. Medium-term: replacing HC-05 with an ESP32 adds Wi-Fi and MQTT cloud telemetry. Long-term: adding a camera and a Raspberry Pi Zero enables semantic obstacle classification — the step from reactive to intent-aware navigation.

Acknowledgment

The authors thank Ms. Rohini Banait (Project Advisor), Ms. Padma Nimbhore (Project Coordinator), and Dr. P. D. Ganjewar (Head, Dept. of Computer Engineering) at MIT Academy of Engineering, Alandi, Pune, for laboratory access and guidance throughout this work.

REFERENCES

- [1] R. Gupta, Robotics and Control. New Delhi: New Age International, 2015.
- [2] S. Jaikaruna et al., "Low-Cost Autonomous Line-Following Cart Using IR Sensors," in Proc. IEEE ICET, 2025, pp. 112-117.
- [3] Y. Jin, Z. Liu, and H. Wang, "Intelligent Obstacle-Avoidance Vehicle Using Rotating Ultrasonic Sensors," in Proc. IEEE ICRAE, 2019, pp. 54-59.
- [4] D. Mukhopadhyay, S. Das, and R. Sarkar, "IoT-Enabled Remote Controlled Car with Bluetooth and Wi-Fi," in Proc. IEEE ICSIT, 2020, pp. 203-208.
- [5] T. L. Floyd, Electronic Devices, 10th ed. Pearson, 2011.
- [6] Arduino, "Arduino Uno Board Overview," 2024. [Online]. Available: <https://www.arduino.cc/en/Main/ArduinoBoardUno>.
- [7] Components101, "HC-SR04 Ultrasonic Sensor Datasheet," 2024. [Online]. Available: <https://components101.com>.
- [8] Components101, "HC-05 Bluetooth Module Datasheet," 2024. [Online]. Available: <https://components101.com/wireless/hc-05-bluetooth-module>.
- [9] Components101, "L298N Motor Driver Module Documentation," 2024. [Online]. Available: <https://components101.com/modules/l298n-motor-driver-module>.
- [10] Components101, "IR Sensor Module Datasheet," 2024. [Online]. Available: <https://components101.com/sensors/ir-sensor-module>.
- [11] Arduino, "Arduino IDE Reference," 2024. [Online]. Available: <https://www.arduino.cc/reference/en/>.
- [12] IEEE Publications, "Autonomous Robotics and Sensor-Based Navigation," IEEE Research Library, 2023.
- [13] M. Mehta and S. Joshi, "A Review of Line-Following Robot Techniques," Int. J. Eng. Res., vol. 9, no. 3, pp. 441-446, 2020.
- [14] C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping," IEEE Trans. Robot., vol. 32, no. 6, pp. 1309-1332, 2016.
- [15] R. Kumar, A. Sharma, and P. Singh, "Comparative Study of Two-Sensor vs. Five-Sensor Line Following Robots," in Proc. IEEE ICCA, 2022, pp. 389-394.
- [16] S. Roy and D. Bhattacharyya, "Wireless Fallback Strategy for Autonomous Rovers Using Watchdog Timers," in Proc. IEEE ICSSS, 2021, pp. 112-116.
- [17] K. J. Astrom and T. Hagglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. ISA, 1995.
- [18] S. Mohan, R. Patel, and V. Mehta, "PID vs. Bang-Bang Control for Mobile Robot Line Following," in Proc. IEEE ICICT, 2023, pp. 78-83.
- [19] M. Fracapane, R. de Koster, F. Sgarbossa, and J. Strandhagen, "Planning and Control of Autonomous Mobile Robots for Intralogistics," Eur. J. Oper. Res., vol. 294, no. 2, pp. 405-426, 2021.
- [20] B. Keselj, M. Braut, and I. Petrovic, "Mobile Robot Navigation in Structured Indoor Environments," in Proc. IEEE MIPRO, 2022, pp. 1040-1045.
- [21] A. Nair, S. Pillai, and V. Kumar, "IoT Robotics Kits in Undergraduate Education: A Case Study," in Proc. IEEE EDUCON, 2023, pp. 1201-1206.
- [22] O. Patil, M. Kulkarni, and S. Desai, "Multi-Sensor Obstacle Detection for Low-Cost Rovers," in Proc. IEEE ICACCI, 2022, pp. 558-563.
- [23] A. Agarwal, P. Verma, and R. Gupta, "Eight-Sensor PID Line Follower with Sub-2 mm Deviation," in Proc. IEEE ICECA, 2024, pp. 311-316.